Ansh Sarfare **D20A - 51**

BC Experiment-6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

What is a Smart Contract? A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

About the Crowdfunding Platform: A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

IMPLEMENTATION:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract CrowdfundingContract {
```

```
    struct Campaign {
        uint256 campaignId;
        string title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

        event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {
        campaignCount++;
        uint256 deadline = block.timestamp + (_durationDays * 1 days);
        campaigns[campaignCount] = Campaign(
            campaignCount,
            _title,
            _goalAmount,
            0,
            deadline,
            payable(msg.sender),
            true
        );
        emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
    }

    function receiveDonation(uint256 _campaignId, uint256 _amount) external {
        Campaign storage c = campaigns[_campaignId];
        require(c.isActive, "Campaign is not active");
        c.raisedAmount += _amount;
        emit FundsReceived(_campaignId, _amount);
    }

    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
```

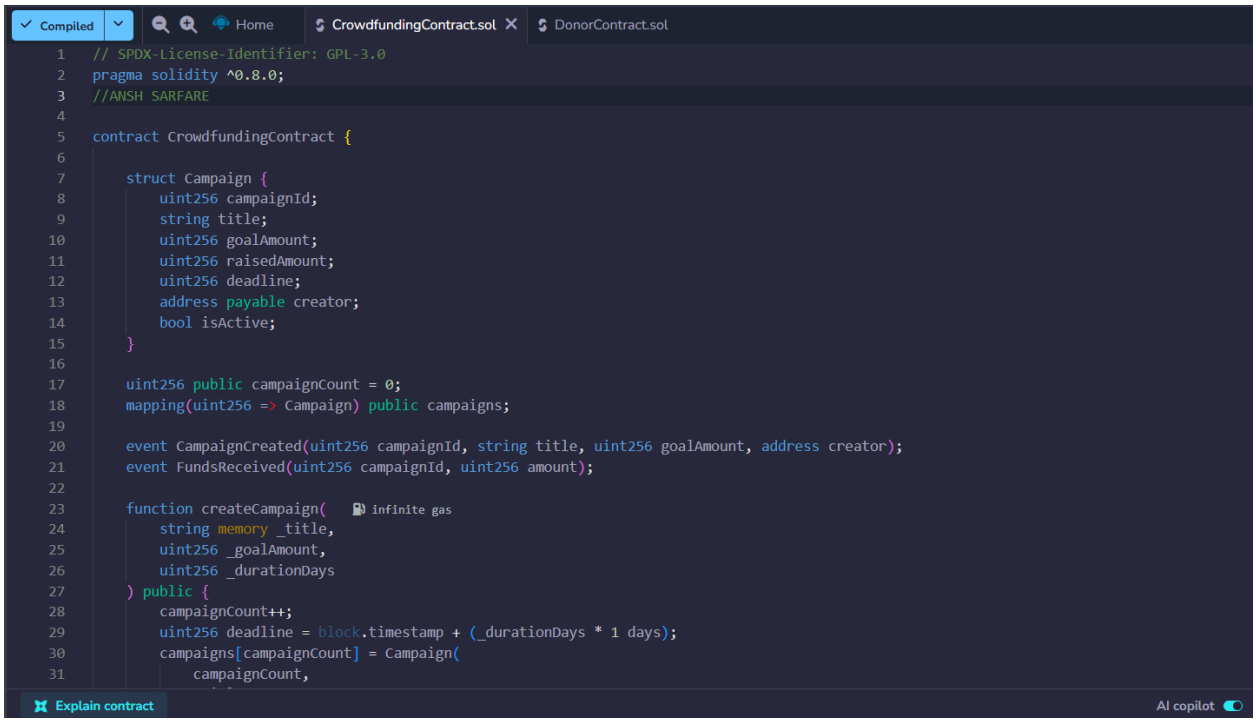
```
    ) {
        Campaign storage c = campaigns[_campaignId];
        return (
            c.campaignId,
            c.title,
            c.goalAmount,
            c.raisedAmount,
            c.deadline,
            c.creator,
            c.isActive
        );
    }

    function closeCampaign(uint256 _campaignId) external {
        campaigns[_campaignId].isActive = false;
    }
}
```

DonorContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}
contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }
    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
```

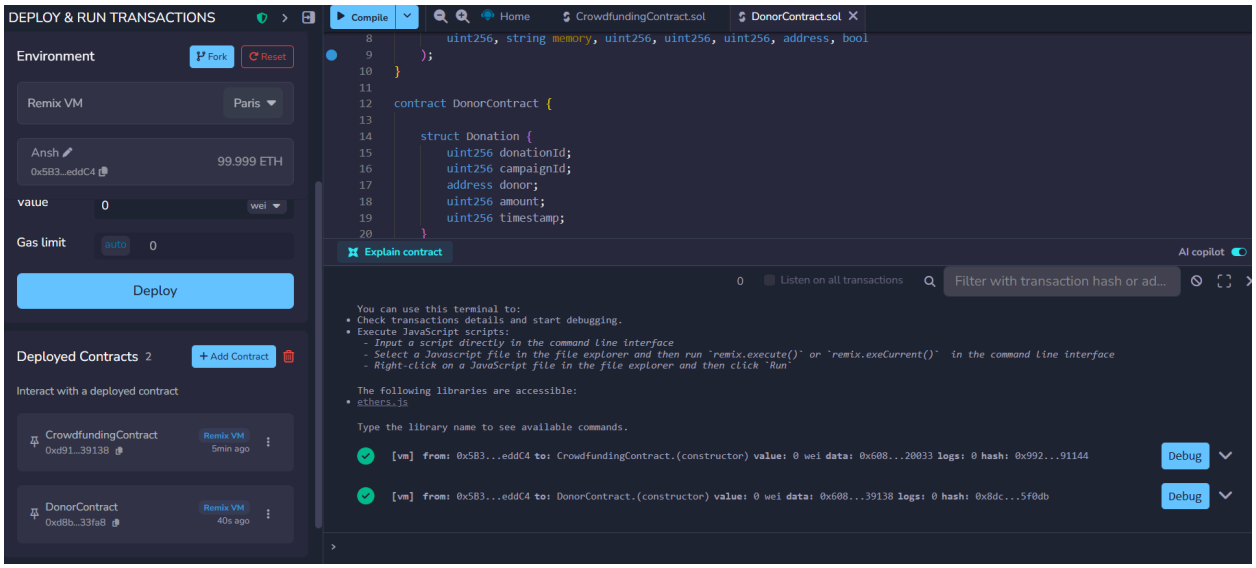
```
    function donate(uint256 _campaignId, uint256 _amount) public {
        donationCount++;
        donations[donationCount] = Donation(
            donationCount,
            _campaignId,
            msg.sender,
            _amount,
            block.timestamp
        );
        ICrowdfundingContract(crowdfundingContractAddress)
            .receiveDonation(_campaignId, _amount);

        emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
    }
    function getDonation(uint256 _donationId) public view returns (
        uint256, uint256, address, uint256, uint256
    ) {
        Donation storage d = donations[_donationId];
        return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
    }
    function getCrowdfundingContract() public view returns (address) {
        return crowdfundingContractAddress;
    }
}
```

Compilation:

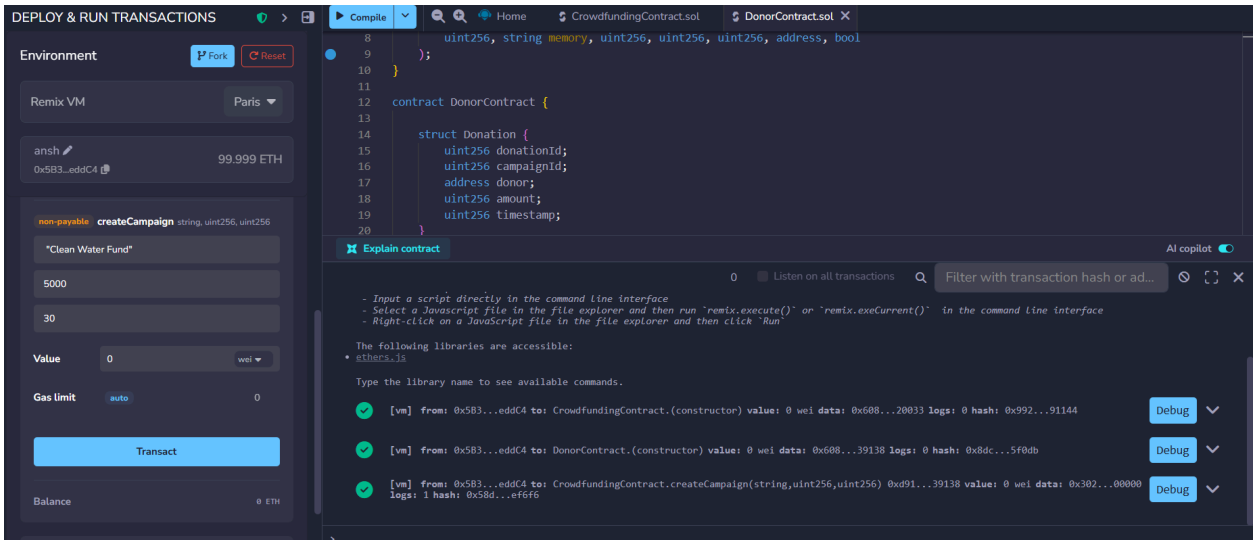


```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//ANSH SARFARE

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;   - gas
    function getCampaign(uint256 _campaignId) external view returns (    - gas
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}

contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }

    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);

    constructor(address _crowdfundingAddress) {    infinite gas 340400 gas
        crowdfundingContractAddress = _crowdfundingAddress;
    }
```

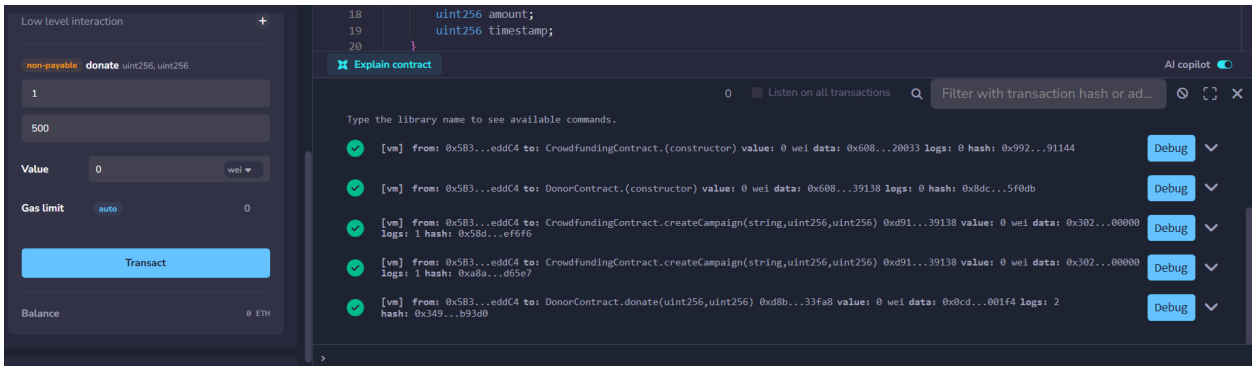
DEPLOYMENT:

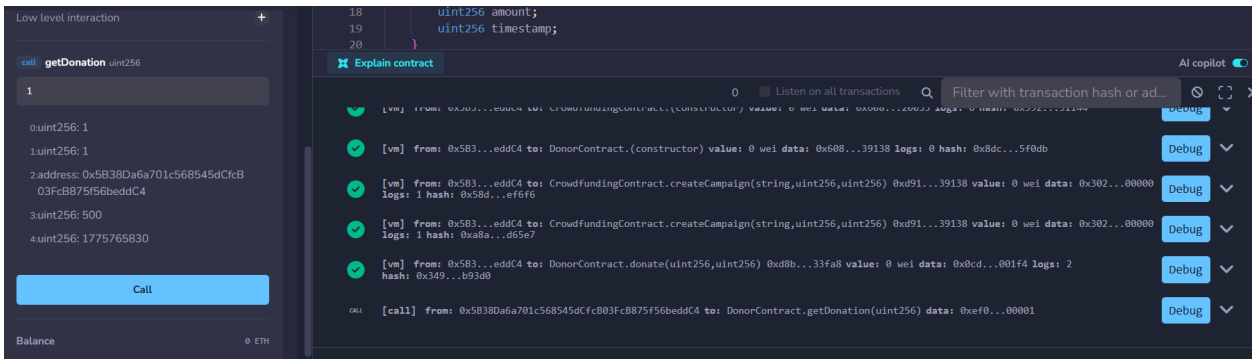


TRANSACTIONS:



DONATIONS





CONCLUSION:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.